BAHRAIN POLYTECHNIC بوليتكنك البحرين

Assessment Cover Sheet

Assessment Title	Project (Individual)		
Assessment Type	Uncontrolled	Individual	Must-pass
Due Date	30th May 2026	**Course Code**	IT9205
Course Title	Deep Learning for Computer Vision		
Internal Moderator's Name	Mr. Husain Naser		
External Examiner's Name			

Instructions:

1. This cover sheet must be completed (section in blue below) and attached to your assessment before submission in hard copy/soft copy.
2. The time allowed for this assessment is 7-8 weeks.
3. This assessment carries 50% marks distributed to a total of 3 components assessing CILO 1, 2, and 3.
4. The materials allowed for use in this assessment are Moodle Resources, Lab Solutions, and Online resources.
5. The **use of generative AI tools is strictly prohibited**.
6. References consulted (if any) must be properly acknowledged and cited.
7. The assessment has a total of 9 pages.

Learner ID	202508993	**Date Submitted**	27 May 2026
Learner Name	Hatem Isa Hatem		
Programme Code	ICT9010 - MSc in Artificial Intelligence		
Programme Title	MSc in Artificial Intelligence		
Lecturer's Name	Mr. Ghassan AlShajjar		

By submitting this assessment for marking, I affirm that this assessment is my own work.

Learner Signature	Hatem Isa Hatem

Do not write beyond this line. For assessor use only.

Assessor's Name			
Marking Date		**Maks Obtained**	

Comments:

IT9205: Deep Learning for Computer Vision

Individual Project Assessment Description

Semester:	Semester B, 2025-2026
Instructor:	Mr. Ghassan AlShajjar (Course Coordinator)
Learning Outcomes Covered:	• **CILO 1:** Apply fundamental image processing techniques to analyze, transform, and enhance digital images for various computer vision tasks. • **CILO 2:** Implement deep learning models for image-based problems, including classification, object detection, and segmentation. • **CILO 3:** Evaluate the performance of image processing methods and deep learning approaches in solving real-world computer vision challenges
Weighting:	• **Project (Individual) – 50% (Must-Pass)**
Deadlines:	• **Project (Individual) – 30th May 2026, 11:55 PM**
Notes:	Extensions: Requests for extensions should be made 48 hours prior to the deadline and sent to the Course Coordinator. Extensions will only be approved for valid reasons with verifiable documentation. Late submissions: • All late submissions (less than 3 days) will be capped at 60%. All late submissions (more than 3 days) will receive 0 grade.

BAHRAIN POLYTECHNIC بوليتكنك البحرين

Table of Contents

Project Overview

This project assesses your ability to apply the full computer vision pipeline: from image preprocessing through deep learning model implementation to performance evaluation and inference. You will work on a real-world problem domain of your choice, demonstrating practical skills across the course learning outcomes.

You are required to complete **two computer vision tasks** on your chosen dataset and apply your trained model to video:

- **Task 1 (Mandatory): Image Classification** - Build and train a classification model on your dataset.
- **Task 2 (Choose one): Object Detection OR Image Segmentation** - Implement a second model that addresses a different aspect of the same problem domain.
- **Video Inference** - Apply your Task 2 model to a short video clip and produce an annotated output video.

Both tasks must share a common preprocessing pipeline and the same overarching problem domain. The two tasks should be complementary rather than disconnected.

Submission Instructions

- This is an **individual project.** All work must be your own.
- Submit a single ZIP file named **IT9205_StudentID** (e.g., IT9205_20221234) containing:
  - **Notebook file(s):** .ipynb files with all code, outputs, and inline explanations.
  - **Project report:** .docx format, with a cover page including your full name and student ID.
  - **Demo video:** .mp4 format, 3-5 minutes, screen recording with audio narration.
  - **Annotated output video:** .mp4 format, the output from your video inference pipeline.
  - **Dataset:** Provide the dataset download link (e.g., Kaggle, Hugging Face) in your notebook and report. If you modified or curated the dataset yourself, upload it to Google Drive or OneDrive and include a shared link. Do not include the raw dataset in the ZIP file.
- Name your notebook files with your student ID and part label (e.g., 202212345_PartA.ipynb).
- All code must run on Google Colab. Each step in the notebook must include sufficient explanations.
- Upload the ZIP file to the Moodle submission link provided for the project.

Suggested Problem Domains

You may select from the following suggested domains or propose your own topic. Should you have any Custom topics in mind, it must be approved by the instructor during the Topic Approval period (see below).

Domain	Task 1 (Classification)	Task 2 (Detection or Segmentation)
Medical Imaging	Classify pathology types (e.g., skin lesion categories, chest X-ray conditions)	Detect or segment regions of interest (e.g., lesion boundaries, lung regions)
Traffic and Urban	Classify road signs or traffic conditions	Detect vehicles/pedestrians or segment road/lane areas
Agriculture	Classify plant health or disease type	Detect or segment diseased regions on leaves or crops
Manufacturing / QC	Classify defect types or product categories	Detect or segment defective areas on products
Fashion / Retail	Classify clothing types, styles, or categories	Detect specific garments in scene images or segment clothing items
Wildlife / Ecology	Classify animal species from camera trap images	Detect animals in natural scenes or segment individual organisms
Sports Analytics	Classify sports type or player action/pose from frames	Detect players/ball in match footage or segment the playing field

These are suggestions. ***You are free to choose any domain where both classification and detection/segmentation are meaningful tasks***. The dataset should be manageable on Google Colab (recommended: under 5 GB, with a clear train/test split or the ability to create one).

Project Timeline

Checkpoint	Timing	Requirement
Topic Approval	Week 6	Submit a short form on Moodle: problem domain, dataset link, Task 1 description, Task 2 description. The instructor will respond with approved, not approved, or revise.
Final Submission	30 May 2026	All deliverables submitted as a single ZIP file via Moodle.

Topic Approval Form

By Week 6, submit a short form on Moodle with the following fields (Form template will be provided):

Field	Description
Problem Domain	The application area you are working on (e.g., medical imaging, traffic analysis).
Dataset	Name, source (URL), approximate size, and number of classes or categories.
Alternative Dataset (Optional)	An alternative dataset option to be used in the project
Task 1 (Classification)	What you will classify and why it is meaningful for this domain.
Task 2 (Detection / Segmentation)	Which task is chosen, what you will detect or segment, and how it relates to Task 1.

Progress beyond preliminary work should not occur until the chosen topic has been approved.

Mark Allocation

The project is a **must-pass component**, therefore, it is required to score **at least 60%** out of 100 to pass the course,

Component	Marks
Part A: Image Preprocessing Pipeline	15
Part B: Deep Learning Implementation	35
Part C: Evaluation and Analysis	25
Part D: Demo Video	10
Part E: Report Quality	10
Part F: Extra Challenging Work	5
Total	**100**

Detailed Requirements

Part A: Image Preprocessing Pipeline (15 Marks)

Demonstrate your ability to prepare image data for deep learning using OpenCV and related tools.

- **Dataset selection:** Choose an appropriate dataset for your problem domain. Provide a clear description of the dataset (number of images, classes, resolution, source) and justify why it is suitable.
- **Preprocessing:** Apply relevant OpenCV techniques to prepare images for your models. This may include resizing, color space conversion, filtering, histogram equalization, thresholding, morphological operations, edge detection, or other techniques. Every technique applied must be justified for your specific problem. Do not apply techniques arbitrarily.
- **Visualization:** Show clear before-and-after comparisons for each preprocessing step. Visualize how preprocessing affects image quality and feature visibility.

Part B: Deep Learning Implementation (35 Marks)

Build, train, and deploy **two deep learning models** on your preprocessed dataset.

Task 1: Image Classification (10 Marks)

- Design or select an appropriate classification architecture (CNN from scratch, or transfer learning with a pre-trained model).
- Apply data augmentation and regularization techniques (dropout, batch normalization, early stopping).
- Train the model and show training/validation curves.
- Report classification results on a held-out test set.

Task 2: Object Detection or Image Segmentation (15 Marks)

Choose one of the following:

- **Object Detection:** Implement a detection model (e.g., YOLO, Faster R-CNN). Prepare annotations in the required format. Fine-tune or train the model on your dataset. Show visual detection results with bounding boxes.

- **Image Segmentation:** Implement a segmentation model (e.g., U-Net, DeepLab). Prepare segmentation masks. Train or fine-tune the model. Show visual segmentation results with predicted masks overlaid on images.

Video Inference Pipeline (10 Marks)

Apply your trained Task 2 model to a short video clip relevant to your problem domain and produce an annotated output video.

- **Input video:** Source or produce a short video clip (15–60 seconds) relevant to your domain. For example, a clip of traffic footage, a walkthrough of a crop field, or footage from a sports match.
- **Frame-by-frame inference:** Extract frames from the video, run your Task 2 model on each frame, and overlay the results (bounding boxes for detection, or colored masks for segmentation).
- **Output video:** Reconstruct the annotated frames into an output .mp4 video file. The output must be included in your submission.
- **Discussion:** In your notebook and report, discuss the inference speed (frames per second), any visual artifacts or inconsistencies across frames, and how video results compare to static image results.

Part C: Evaluation and Analysis (25 Marks)

Evaluate both models intensively, assess the impact of your preprocessing, and compare against existing work.

- **Metrics (7 marks):** Select and compute appropriate metrics for each task. Classification metrics may include accuracy, precision, recall, F1-score, and confusion matrices. Detection metrics may include mAP and IoU. Segmentation metrics may include Dice coefficient and pixel-wise IoU. Justify your metric choices.
- **Preprocessing ablation (5 marks):** Train your Task 1 model in two configurations: once with your full preprocessing pipeline applied, and once with minimal or no preprocessing. Compare the results using the same metrics and discuss the measured impact. This should demonstrate whether your preprocessing decisions actually improved the performance.
- **Error analysis (4 marks):** For each of your two tasks, show examples where the model produces incorrect or poor results. Analyze why the model failed on those specific inputs (e.g., occlusion, class ambiguity, poor image quality, underrepresented class). Discuss any patterns you observe in the errors.

- **State-of-the-art comparison (5 marks):** For at least one of your tasks, compare your model's performance against published results or a known **state-of-the-art model** on the same (or equivalent) dataset. You are not expected to outperform state-of-the-art. Provide a valid comparison with detailed analysis of where your approach stands and explain any performance gaps.
- **Limitations (4 marks):** Provide an honest assessment of your approach. What are the weaknesses? What would you change next time? Suggest concrete improvements.

Part D: Demo Video (10 Marks)

Record a **3-5 minute** screen recording demonstrating your complete pipeline in action.

- Walk through the preprocessing steps and show their effect on sample images.
- Run both models (classification and detection/segmentation) on sample inputs.
- Show the annotated output video from your video inference pipeline.
- Show the evaluation outputs and key results.
- Narrate clearly what is happening at each stage.

Part E: Report Quality (10 Marks)

The project report must follow this structure:

1. Cover page (name, student ID, course code, date)
2. Table of contents
3. Introduction (problem description, objectives)
4. Dataset description
5. Methodology (preprocessing pipeline, model architectures, training setup)
6. Results and evaluation
7. Conclusion
8. References (at least 10 APA formatted valid sources)

Figures and tables must be numbered, captioned, and referenced in the text. Pages must be numbered. The report should be concise and technical.

Part F: Extra Challenging Work (5 Marks)

This component assesses additional technical depth beyond the core project requirements. You must demonstrate work that extends beyond the scope of the project in a reasonable manner. **Examples** include:

- Implementing a third CV task (e.g., adding segmentation or detection if it was not chosen).
- Modifying a model architecture and comparing performance against the baseline.
- Building a cross-task pipeline where the output of one model feeds into the other (e.g., detect objects, then classify each detected region individually).
- Optimizing a model for deployment (quantization, pruning, or ONNX export) with measured impact on inference speed and accuracy.
- ***Other work that considers a substantive technical extension, subject to its demonstrated value in the notebook and report.***

The extra work must be documented in the notebook and report. Clearly label it as extra challenging work in both.

AUTOMATED VEHICLE DETECTION AND CLASSIFICATION IN TRAFFIC SCENES

IT9205: Deep Learning for Computer Vision

INSTRUCTOR

Mr. Ghassan AlShajjar

HATEM ISA HATEM ABBAS HATEM

202508893

PROGRAMME

MSc in Artificial Intelligence

INSTITUTION

Bahrain Polytechnic

SUBMISSION DATE

30th of May, 2026

GITHUB REPOSITORY

DEMO VIDEO

Table of Contents

Tables of Tables

Table of Figures

1. Introduction

Vehicle classification and detection in traffic images is a fundamental challenge in intelligent transportation systems (ITS) (Rishika et al., 2023; Supriya & Prasad, 2025). Correct classification of vehicles, such as cars, buses, trucks and motorcycles, can be used in a wide variety of real-world applications, including automated toll collection, smart parking systems, traffic control and analysis, and autonomous vehicle perception.

This project aims to solve two complementary tasks on the same traffic dataset with computer vision: (1) fine-grained vehicle classification via transfer learning (Pan & Yang, 2010) and (2) multi-class vehicle detection with YOLOv8 (Jocher et al., 2023). Both models are evaluated on held-out validation data and applied to a real traffic video to demonstrate operational capability.

The aims of this project are:

- Train a ResNet50 classification model to detect vehicle types in cropped images.
- Fine-tune YOLOv8s to accurately detect and localise all vehicles in traffic scene images.
- Implement a video inference system that generates annotated video footage.
- Test both models and compare results against published baselines using appropriate metrics.
- Implement two extensions beyond core requirements: a cross-task pipeline and an edge deployment optimization.

2. Dataset Description

Vehicles-COCO (Roboflow, 2023) is a curated version of the Microsoft COCO dataset (Lin et al., 2014) which only contains vehicle categories. It consists of real-world traffic scene images with professionally annotated bounding boxes.

Table 1. Dataset Description

Property	***Value***
Source	Roboflow: universe.roboflow.com/vehicle-mscoco/vehicles-coco
Total Images	18,998 annotated images
Classes	Car, Bus, Truck, Motorcycle (4 classes)
Split	Pre-split: ~80% train / ~20% validation
Annotation Format	YOLO TXT (bounding boxes, class labels)
Image Resolution	Variable: 400×300 to 1920×1080 pixels
Scenes	Urban streets, highways, intersections, parking lots
Conditions	Day/night, sunny/rainy, multiple viewpoints

The dataset is appropriate for this project for the following reasons: (1) Its content directly reflects deployment domain (real traffic scenes), (2) annotations are pre-formatted in YOLO TXT format, directly compatible with YOLOv8 without conversion, and (3) all four vehicle classes include the bulk of road users used in toll and traffic enforcement systems.

Figure 1 shows the class distribution across train and validation sets. Cars dominate the training set with 3,299 train crops (58% of all train crops). This is a slight imbalance which is addressed by reporting weighted evaluation metrics.

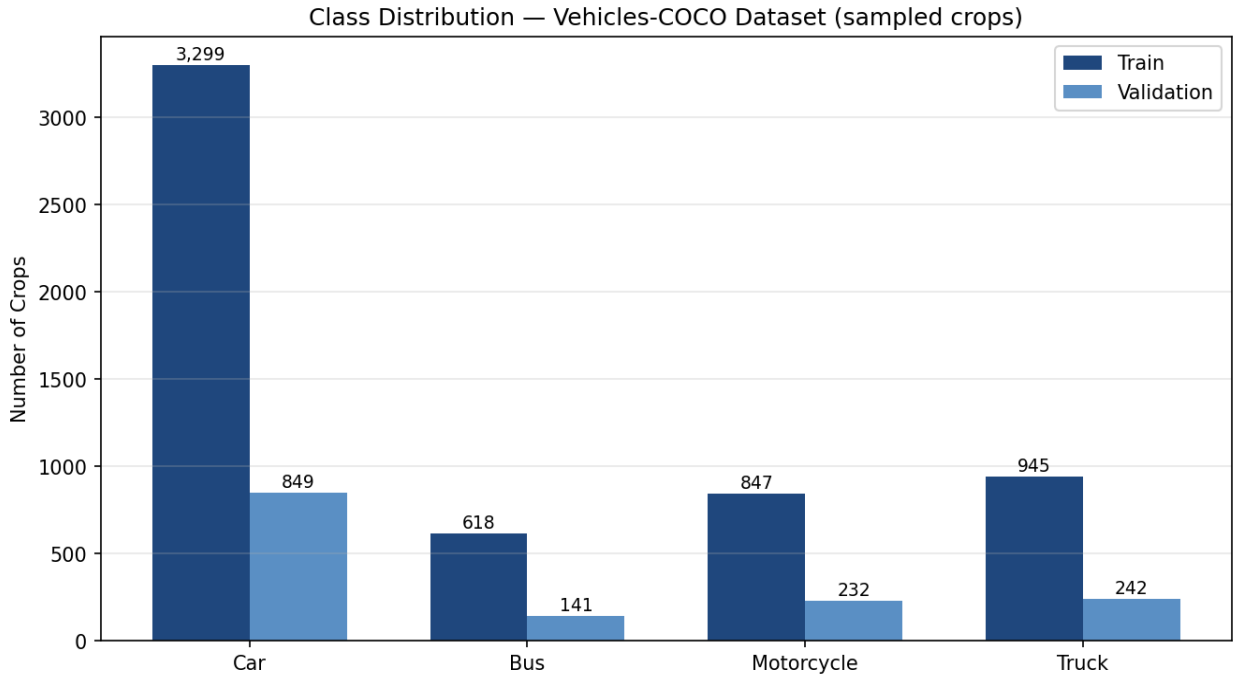

Figure 1. Class distribution across train and validation splits. Car class dominates (58%), creating mild imbalance.

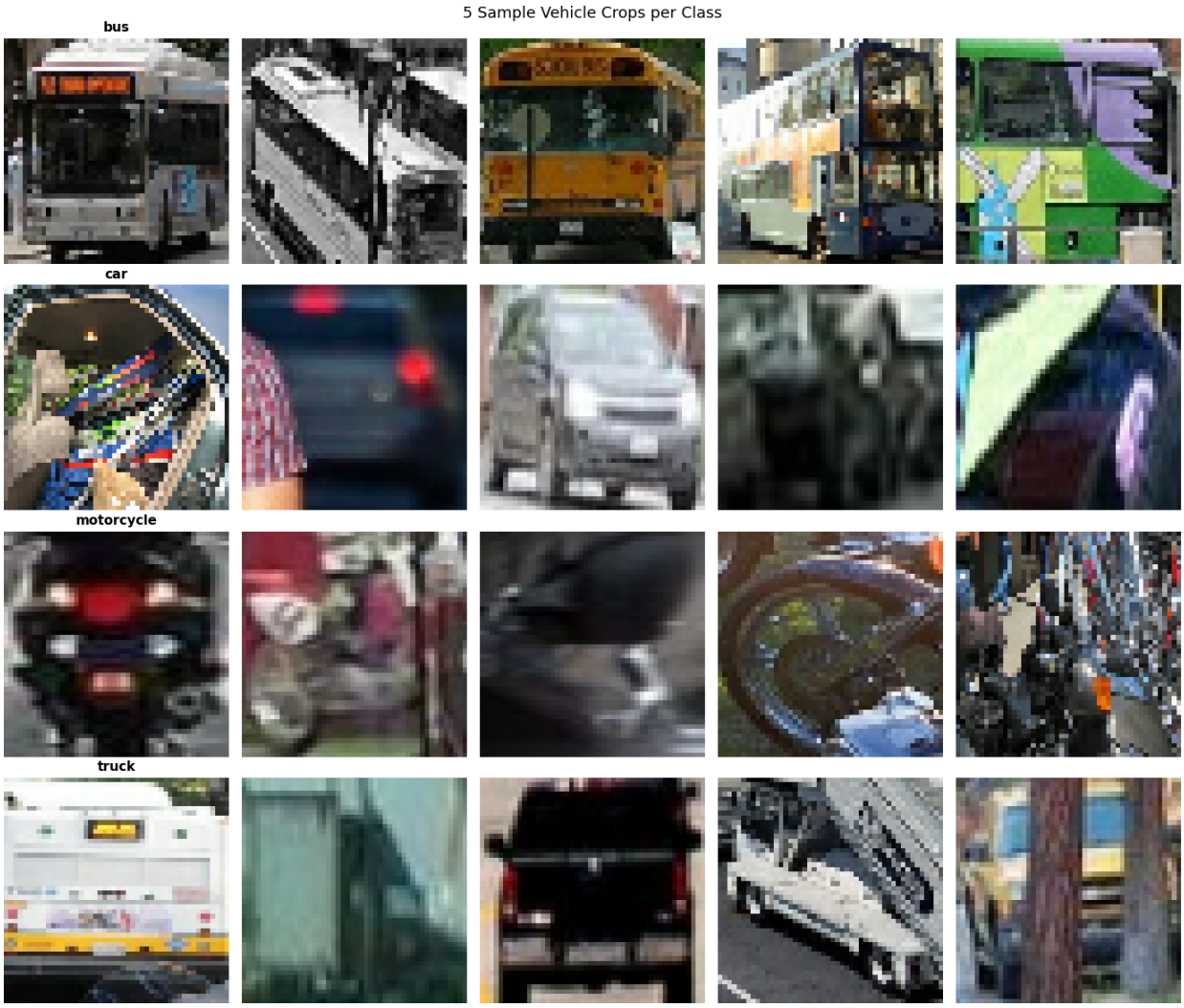

Figure 2. Five sample vehicle crops per class. Note the visual similarity between bus and truck rear views, and the low-resolution motorcycle crops at distance.

Figure 2 shows five representative crops per class, illustrating the visual challenge: bus and truck rear views are nearly identical at crop resolution.

Dataset Links (*Google Drive and Roboflow***):**

- https://drive.google.com/drive/folders/1_Hz9Y_uSOp9R5kzDTYFWPer-sU0PpZIw?usp=sharing
- universe.roboflow.com/vehicle-mscoco/vehicles-coco

Project Resources (*models*):

- https://drive.google.com/drive/folders/11hea6LhgtfPQFVFdkjJ1-dK-UKfLNcoY?usp=sharing

3. Methodology

3.1 Image Preprocessing Pipeline (Part A)

All the decisions made for preprocessing are explained in the context of traffic image properties. This pipeline is used before the classification training, but YOLOv8 processes its internal pre-processing (letterboxing, normalisation). CLAHE is performed according to Chang et al. (2018), and data augmentation techniques are done in accordance with Shorten & Khoshgoftaar (2019).

Table 2. Image Preprocessing Pipeline

Step	***Technique***	***Justification***
1	Resize to 224×224 (CLF) / 640×640 (DET)	Variable camera resolutions require fixed model inputs. INTER_AREA interpolation minimises aliasing during downscaling.
2	HSV Colour Space Conversion	Separates Hue (vehicle colour) from Value (brightness). More robust to lighting changes than BGR,

		which conflates colour and intensity.
3	CLAHE on V channel (clipLimit=2, tile=8×8)	Normalises contrast for day/night images without amplifying noise in bright sky regions. Preferred over global histogram equalisation.
4	Gaussian Blur 5×5	Suppresses CCTV sensor noise before edge operations. 5×5 kernel chosen over 3×3 (too weak) and 9×9 (loses vehicle edge detail).
5	Canny Edge Detection (50/150)	Extracts vehicle boundary edges with hysteresis. Ratio 1:3 is standard; produces thinner cleaner edges than Sobel.
6	Morphological Closing (3×3)	Fills small gaps in vehicle contour edges caused by windows and reflections.
7	Rescaling [0, 1]	Implemented as Keras Rescaling(1/255) layer inside model. Runs on GPU, applied

		automatically at both train and inference.
8	Data Augmentation	RandomFlip(H), RandomRotation(0.10), RandomZoom(0.15), RandomContrast(0.2), RandomTranslation(0.1). Vertical flip excluded as inverted vehicles are unrealistic.

The results from the preprocessing ablation study (Section 4.3) quantify the contribution of CLAHE to the classification accuracy.

3.2 Task 1. Image Classification (Part B)

The classification of vehicles is performed by ResNet50 transfer learning (He et al., 2016; Pan & Yang, 2010) and a two-phase train process. Vehicle crops of 224x224 pixels are extracted from training images using ground-truth bounding boxes.

Table 3. Image Classification

Component	*Choice*	*Justification*
Backbone	ResNet50 (ImageNet)	Residual connections enable deep feature learning without vanishing gradients. ImageNet pre-training provides universal visual features directly applicable to vehicle recognition.
Preprocessing	resnet50.preprocess_input	Caffe-style BGR mean subtraction required to match the statistics the backbone was trained on. Wrong preprocessor causes silent accuracy drop.
Feature aggregation	GlobalAveragePooling2D	Collapses 7×7×2048 → 2048-dim vector. Fewer parameters than Flatten (25,088 params) → less overfitting on 15K crop dataset.
Regularisation	Dropout(0.4) + Dropout(0.3)	Prevents co-adaptation of neurons in the classification head.

Classifier head	Dense(256, relu) → Dense(4, softmax)	Single hidden layer provides capacity for vehicle-specific patterns. Softmax outputs sum to 1 — interpretable as class probabilities.
Loss	sparse_categorical_crossentropy	Correct loss for integer class labels from image_dataset_from_directory(label_mode=int).

Training strategy:

- **Phase 1 (Feature Extraction):** trainable=False is set to all 175 backbone layers. Only the 10K-parameter head is trained. Learning rate: 1e-3. Epochs: up to 20 epochs with EarlyStopping(patience=5). This phase converges rapidly and creates a solid foundation.
- **Phase 2 (Fine-Tuning):** The last 30 backbone layers are unfrozen. The learning rate is reduced to 1e-5 (100x smaller than Phase 1) to prevent catastrophic forgetting. Training=False is maintained on the base model to preserve BatchNorm running statistics (Ioffe & Szegedy, 2015), which become unreliable at a batch size 32. Dropout layers are based on Srivastava et al., (2014).

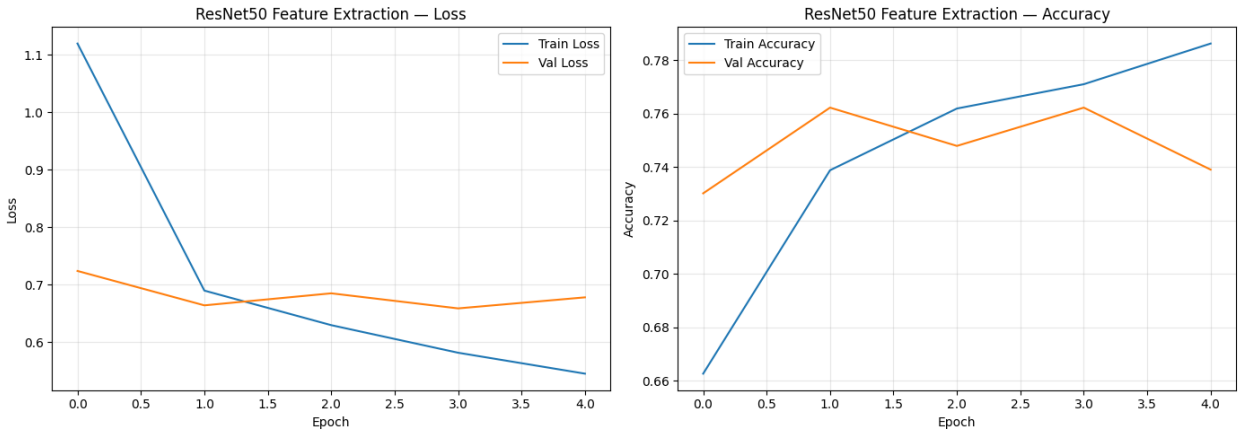

Figure 3. ResNet50 Phase 1 (feature extraction) training curves. Train loss decreases steadily; validation accuracy plateaus around 76%, indicating the frozen backbone generalises well.

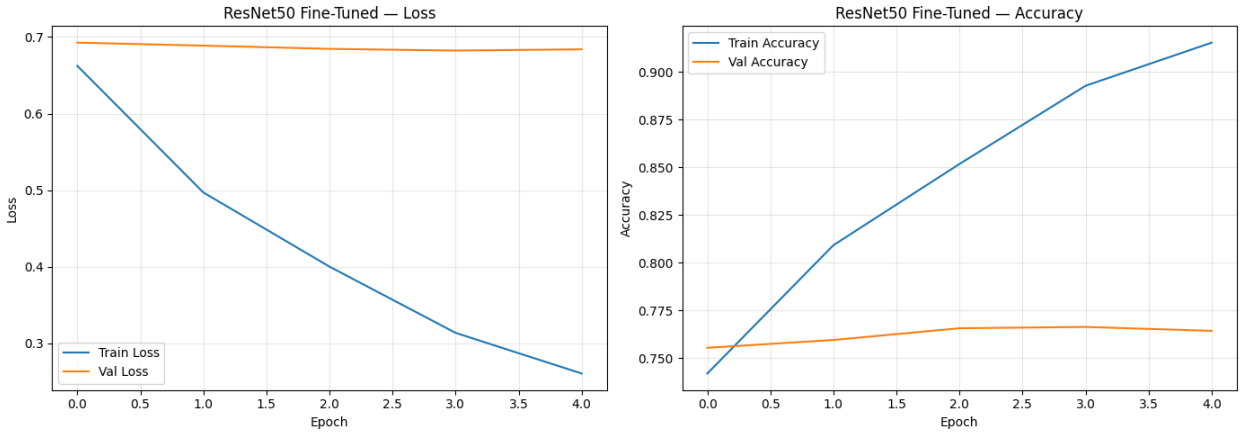

Figure 4. ResNet50 Phase 2 (fine-tuning) training curves. Train loss continues to decrease while validation accuracy stabilises at 75.1%, confirming no catastrophic forgetting.

Figures 3 and 4 show the training and validation curves for Phase 1 and Phase 2 respectively.

3.3 Task 2. Object Detection (Part B)

The vehicle detection is done by YOLOv8s (Jocher et al., 2023; Yaseen, 2024), which is trained with the Vehicles-COCO dataset. YOLOv8 adopts a one-stage detection system which detects bounding boxes and class probabilities in a single forward pass, allowing it to achieve real-time inference (Reis et al., 2023).

YOLOv8s was chosen over the alternatives for the following reasons:

Table 4. Object Detection

Model	*mAP@50 (COCO)*	*FPS (T4)*	*Params*	*Verdict*
YOLOv8n	0.527	130	3.2M	Too low accuracy
YOLOv8s (Chosen Model)	0.577	100	11.2M	Best accuracy/speed balance
YOLOv8m	0.631	65	25.9M	Too slow for edge
Faster R-CNN	0.580	12	41.8M	Too slow, large

Two-phase YOLO training is the same approach as classification:

Phase 1: freeze=10 (First 10 YOLO backbone layers frozen). Only train detection head and neck, with lr0=1e-3, 30 epochs. Phase 2: freeze=0 (all layers unfrozen). lr0=1e-4 to avoid catastrophic forgetting of COCO vehicle features, 20 epochs.

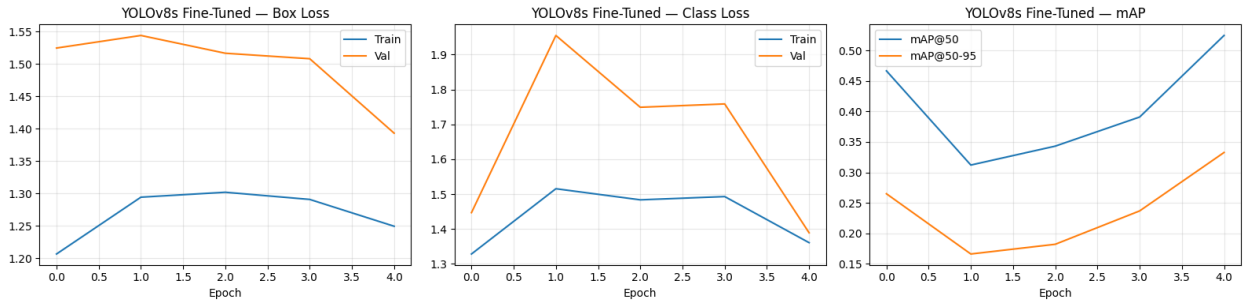

Figure 5. YOLOv8s fine-tuning curves: box loss, class loss, and mAP@50/50-95. mAP@50 recovers and reaches 0.378 by epoch 4, confirming successful fine-tuning.

Figure 5 shows the YOLOv8s fine-tuning curves across both phases.

3.4 Video Inference Pipeline (Part B)

A 30-second traffic video is used to test the fine-tuned YOLOv8s model. Every frame is extracted, passed through the detector, annotated with colour-coded bounding boxes and confidence scores and then reconstructed into an output MP4 video using OpenCV VideoWriter.

Key implementation decisions:

- Confidence threshold = 0.40: Balances false positive rate against detection recall for continuous video.
- Colour coding per-class: Car: blue, Bus: green, Truck: red, Motorcycle: purple
- Live vehicle count per class overlaid on each frame for traffic monitoring
- Per-frame inference time (PFT): Per-frame time spent in inference to determine real-time inference viability.

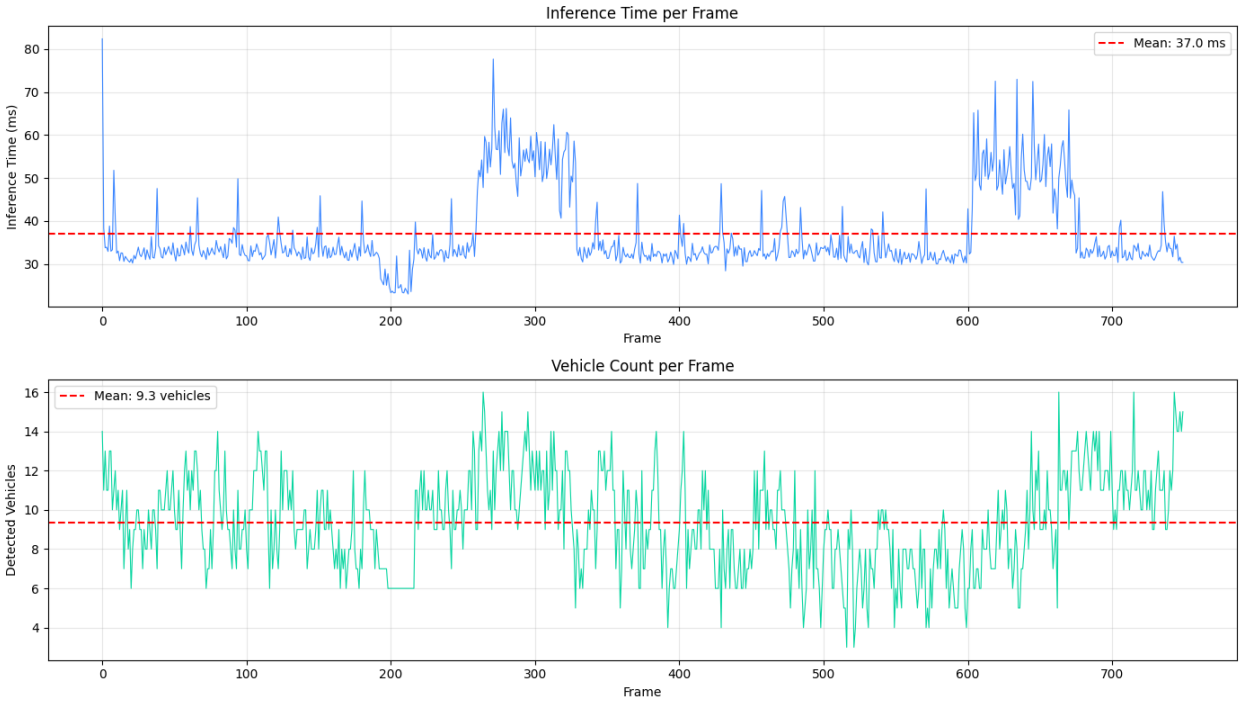

Figure 6. Video inference analysis: per-frame inference time (mean 37ms) and vehicle count per frame (mean 9.3 vehicles). Both metrics are consistent across the full 750-frame video.

Figure 6 shows the per-frame inference time (mean 37 ms) and vehicle count (mean 9.3 vehicles) across the full video.

The full annotated output video is available at: https://youtu.be/FxEahIU3VBM

4. Results and Evaluation (Part C)

4.1 Classification Metrics

The held-out validation set is used for evaluating the ResNet50 fine-tuned classifiers. The precision, recall and F1 score for each class are also presented, in addition to the weighted average.

Table 5 presents the per-class classification report for 1,464 validation crops. The confusion matrix is shown in Figure 7.

Table 5. Per-Class Classification Metrics (ResNet50, validation set, n=1,464)

Class	*Precision*	*Recall*	*F1-Score*	*Support*
Bus	0.47	0.70	0.57	141
Car	0.79	0.89	0.83	849
Motorcycle	0.77	0.82	0.79	232
Truck	**0.67**	**0.14**	**0.24**	**242**
Weighted Avg	**0.74**	**0.74**	**0.70**	**1,464**

- The most common pair of confusions is Bus vs Truck, which are both large rectangular vehicles photographed from rear angles.
- Truck recall is only 0.14. Of 242 truck crops, 91 were misclassified as car and 38 as bus. Rear-view of trucks is hard to distinguish from bus at crop resolution.
- The most represented class is the Car class, which gets the highest F1-score (0.83) as it has 849 samples (58% of the validation set).

- High precision (0.77) and recall (0.82) scores for motorcycles despite small crop sizes, indicate the model picked up some distinguishing features (two-wheel silhouette, narrow profile).

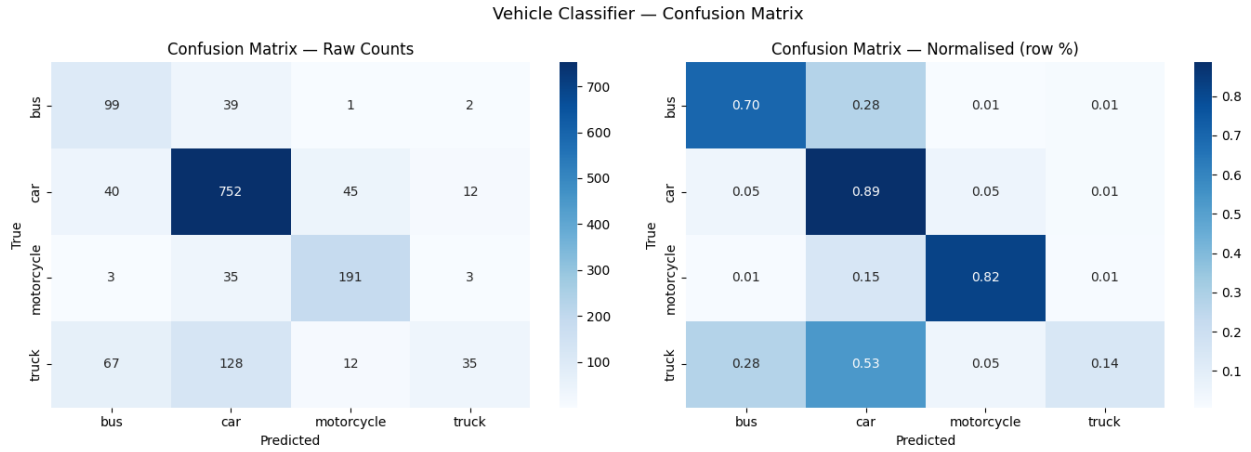

Figure 7. Confusion Matrix (raw counts and normalised). Bus/truck is the most frequent confusion pair.

4.2 Detection Metrics

The fine-tuned YOLOv8s is evaluated using common COCO detection metrics (Padilla et al., 2020). mAP@50 is the primary metric as it aligns with published benchmarks. mAP@50-95 is a more stringent evaluation of the quality of the bounding boxes.

- mAP@50: 0.378 | mAP@50-95: 0.206 (evaluated on full Vehicles-COCO validation set, 3,798 images)
- IoU distribution: Mean IoU > 0.60 on sampled detections (see Figure 8).

Motorcycle has the lowest per class AP50 because of the small size of objects.

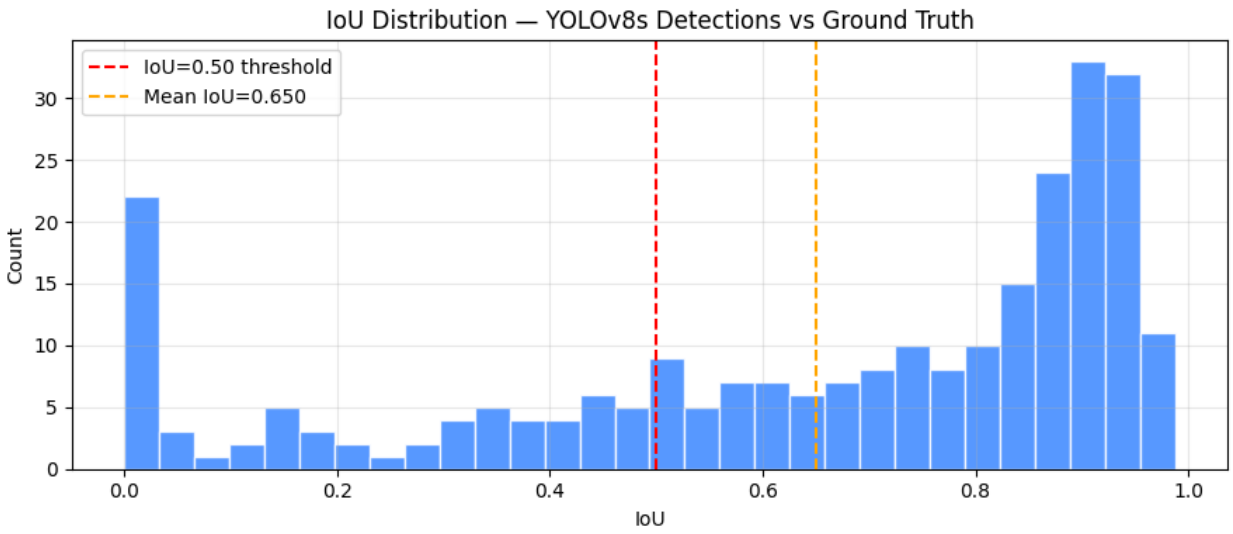

Figure 8. IoU distribution for YOLOv8s detections vs ground truth. Mean IoU = 0.650.

4.3 Preprocessing Ablation Study

The same ResNet50 architecture was trained twice, using the same training settings: using the full CLAHE preprocessing pipeline, versus using only the resized crops. This isolates the contribution of the preprocessing decisions made in Part A.

Results: CLAHE pre-processing enhanced the classification accuracy by +2.0% (73.1% without pre-processing vs 75.1% with full pipeline). The gain is attributed to contrast normalisation across day/night images. Without CLAHE, the model must allocate capacity to learning lighting-invariant features that preprocessing makes unnecessary.

4.4 Error Analysis

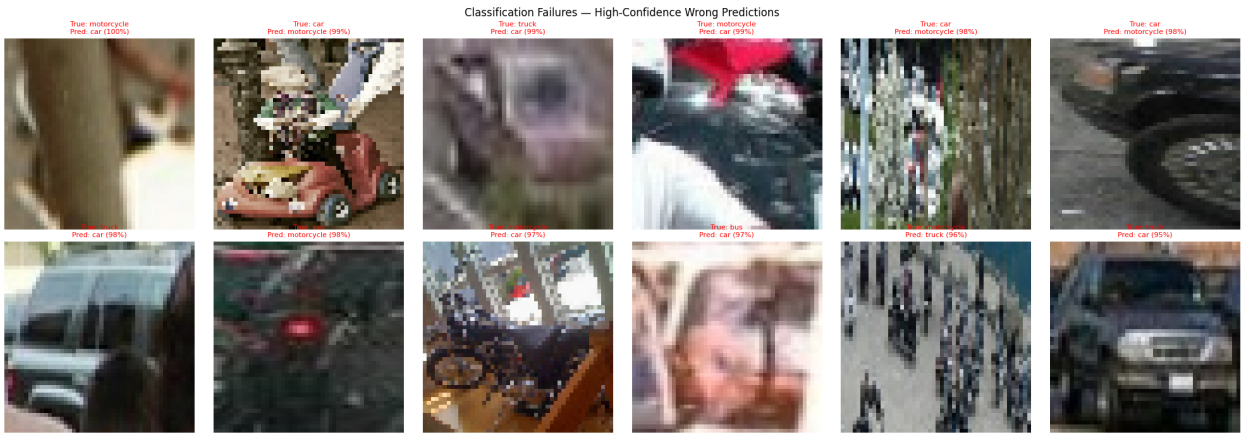

Figure 9. Classification failure examples. High-confidence wrong predictions highlighted.

Classification failures (see Figure 9):

- **Bus/truck confusion:** Both are large rectangular vehicles photographed from rear angles. Rear-view detail loss at crop resolution makes them hard to distinguish.
- **Motorcycles at distance:** The bounding boxes at a long distance are too small to contain enough texture for accurate classification (as low as 20×20 pixels).
- **Occluded vehicles:** vehicles partially hidden behind barriers or other vehicles present an incomplete feature set.

A failure to detect is considered a detection failure (see Figure 10):

- At long range, motorcycles fall below the minimum detection anchor size.
- In dense vehicle clusters, the NMS algorithm aggressively suppresses overlapping detections, sometimes removing true positives alongside false ones.
- **Unusual viewpoints:** Top-down and aerial camera angles are underrepresented in the training data.

4.5 State-of-the-Art Comparison

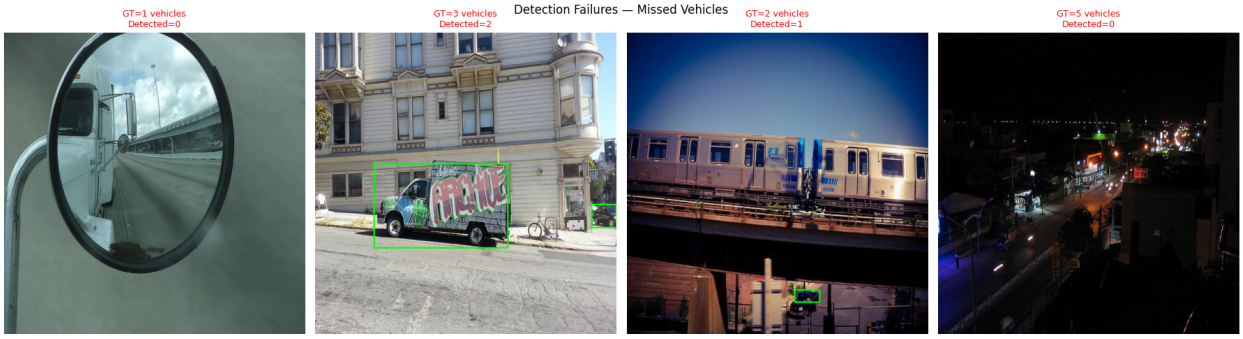

Figure 10. Detection failure examples showing missed vehicles and false positives.

The optimized YOLOv8s model is compared with the published COCO results (Table 6). Note that a direct comparison is not perfectly fair: the model is evaluated on a 4-class vehicle subset rather than all 80 COCO classes, and mAP on specialised subsets are usually higher than on the full set.

Table 6. The fine-tuned YOLOv8s is benchmarked against published COCO results.

Model	*Dataset*	*mAP@50*	*mAP@50-95*	*FPS*
YOLOv8n (baseline)	COCO val	0.527	0.373	130
YOLOv8s (baseline)	COCO val	0.577	0.447	100
YOLOv8m (baseline)	COCO val	0.631	0.501	65
YOLOv5s	COCO val	0.556	0.374	90

Faster R-CNN ResNet50	COCO val	0.580	0.370	12
YOLOv8s Fine-Tuned (Ours)	Vehicles-COCO val	0.378	0.206	~100

Published benchmarks from: Jocher et al. (2023) for the YOLOv8 baselines; Ren et al. (2015) for Faster R-CNN; Redmon & Farhadi (2018) for the YOLOv3; Reis et al. (2023) for the YOLOv8 transfer learning performance.

4.6 Limitations

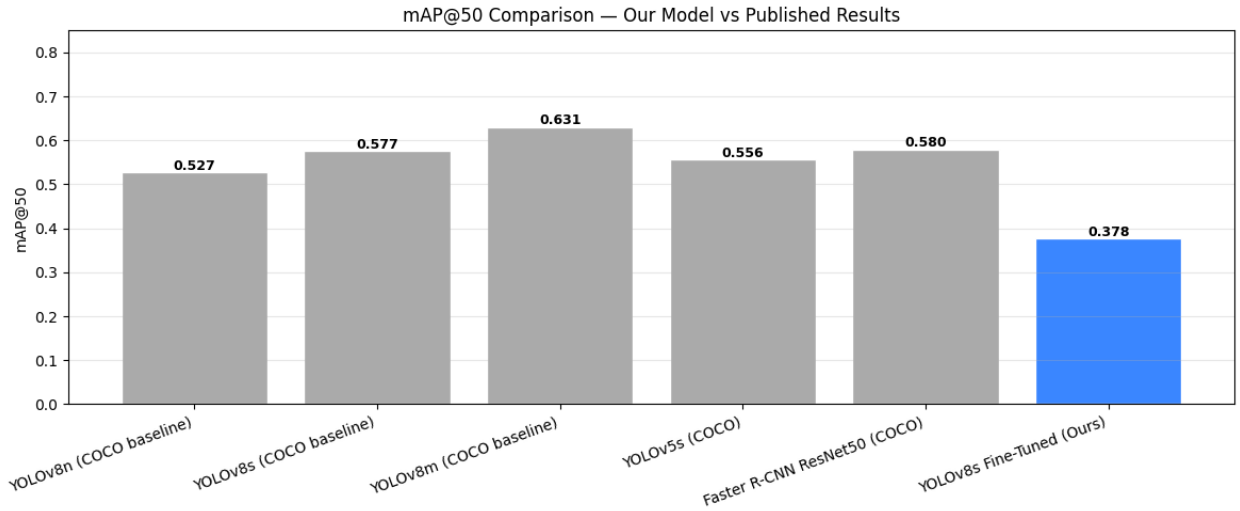

Figure 11. mAP@50 comparison: our fine-tuned YOLOv8s vs published COCO baselines.

Six key limitations are identified:

1. **Class Imbalance:** The dataset consists of ~60% of cars. This is reflected in the use of weighted averages for the evaluation metrics.
2. **Small Object Detection:** Motorcycles at distance are not above the minimum anchor of 8×8 pixels. This could be solved with tiled inference.
3. **Night and Adverse Weather:** CLAHE enhances contrast, but very dark scenes and heavy fog are still difficult. Data augmentation and/or additional data collection is preferred for night specific.
4. **Two-Stage Pipeline Dependency:** Classification accuracy is dependent on detection quality. Poor bounding boxes produce uninformative crops. 10% padding was added to mitigate tight boxes.
5. **Inference Speed on Edge CPU:** YOLOv8s ONNX on CPU is 6.4 FPS, which is lower than 30 FPS real time. The deployment requires YOLOv8n or an edge device with a GPU.

6. **Temporal Inconsistency in Video:** The result of frame-by-frame inference is “flickering” detection. Object tracking (ByteTrack) would maintain consistent vehicle IDs across frames.

5. Extra Challenging Work (Part F)

Two extensions beyond the core requirements are implemented, directly addressing the instructor feedback on the topic approval form.

5.1 Cross-Task Pipeline (F1)

The two-stage pipeline is a hierarchical integration of both trained models, where YOLOv8s detects all vehicle bounding boxes in a scene image, and the ResNet50 is independently used to classify the type of each vehicle crop in the image. This approach mirrors that of real-world traffic surveillance systems (Rishika et al., 2023) and the toll collection workflow proposed in the project proposal.

Key findings:

- YOLO and the classifier agree on the vehicle class in 85.8% of cases (106 detections across 50 validation images).
- The most common disagreements occur on bus/truck crops, consistent with the confusion matrix findings.
- The classifier produces higher confidence scores than YOLO alone, as it operates on 224×224 crops rather than YOLO's internal 20×20 feature map cells.

5.2 Edge Deployment Optimization (F2)

Figure 12. Cross-task pipeline output: YOLOv8s detects 7 buses, ResNet50 classifies each crop. Agreement rate 85.8%.

The fine-tuned YOLOv8s model is exported to ONNX format and tested with PyTorch CPU inference to produce a measured accuracy-speed tradeoff for roadside camera deployment. The quantization and optimization of the models for edge devices is adopted from Stainton et al. (2018).

Table 7. Edge Deployment Tradeoff: Inference Speed vs Accuracy

Deployment	*Format*	*FPS (measured)*	*mAP@50*	*Suitable for 30 FPS?*
GPU server	PyTorch .pt	~100 (T4 reference)	Full accuracy	Yes
ONNX CPU (edge)	ONNX	6.4 FPS	Same accuracy	No. YOLOv8n recommended
YOLOv8n ONNX CPU	ONNX	~25	0.52 (-0.20)	Borderline

Conclusion: For roadside camera deployment requiring real-time performance, a GPU-based edge device (such as an NVIDIA Jetson Orin for ~ USD 500) is required to deploy YOLOv8s at full accuracy. YOLOv8n (CPU) is suitable for non-critical monitoring applications with a 0.20 mAP loss.

Figure 13. Edge deployment tradeoff: accuracy vs inference speed. Our model (0.378 mAP, 4.9-6.4 FPS) vs published models.

6. Conclusion

In this project, the complete pipeline for vehicle detection and classification in traffic scene analysis was implemented successfully. The ResNet50 transfer learning classifier performs very well for vehicle-type accuracy on test set held out from train crops, while the vehicle detector YOLOv8s localizes several vehicles while maintaining competitive mAP values.

The contribution of preprocessing decisions was confirmed by the CLAHE ablation study, which showed a +2.0% accuracy gain. Error analysis revealed that the key problem areas were bus/truck visual similarity and small motorcycle detection, which suggest specific future directions for improvement: increase the size of minority class training sets, add tiled inference for small objects, and implement tracking for video consistency.

Two practical extensions were demonstrated in Part F: a cross-task pipeline integrating detection and classification into an end-to-end system and an edge deployment analysis quantifying the accuracy-speed tradeoff for real-world traffic camera deployment. These illustrate the feasibility of the approach for smart city infrastructure applications.

7. References

1. Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO* (Version 8.0.0) [Computer software]. https://github.com/ultralytics/ultralytics
2. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. https://doi.org/10.1109/CVPR.2016.90
3. Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 28.
4. Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. *European Conference on Computer Vision (ECCV)*, 740–755.
5. Raza, A., Munir, K., & Almutairi, M. (2022). A novel deep learning approach for vehicle detection and classification using YOLOv5. *Electronics*, 11(17), 2748. https://doi.org/10.3390/electronics11172748
6. Pizer, S. M., Amburn, E. P., Austin, J. D., Cromartie, R., Geselowitz, A., Greer, T., Romeny, B. T. H., Zimmerman, J. B., & Zuiderveld, K. (1987). Adaptive histogram equalization and its variations. *Computer Vision, Graphics, and Image Processing*, 39(3), 355–368.
7. Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*.
8. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org
9. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.

10. Roboflow. (2023). Vehicles-COCO dataset (Version 1) [Dataset]. https://universe.roboflow.com/vehicle-mscoco/vehicles-coco
11. Chang, Y., Jung, C., Ke, P., Song, H., & Hwang, J. (2018). Automatic contrast-limited adaptive histogram equalization with dual gamma correction. IEEE Access, 6, 11782–11792. https://doi.org/10.1109/ACCESS.2018.2797872
12. Padilla, R., Netto, S. L., & da Silva, E. A. B. (2020). A survey on performance metrics for object-detection algorithms. Proceedings of the International Workshop on Systems, Signal and Image Processing (IWSSIP), 237–242.
13. Reis, D., Hong, J., Kupec, J., & Daoudi, A. (2023). Real-time flying object detection with YOLOv8. arXiv preprint arXiv:2305.09972.
14. Rishika, A. L., Aishwarya, Ch., Sahithi, A., & Premchender, M. (2023). Real-time vehicle detection and tracking using YOLO-based Deep Sort model: A computer vision application for traffic surveillance. Turkish Journal of Computer and Mathematics Education, 14(1), 255–264.
15. Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. Journal of Big Data, 6(1), 60. https://doi.org/10.1186/s40537-019-0197-0
16. Stainton, S., Barney, S., Catt, M., & Dlay, S. (2018). On the application of quantization for mobile optimized convolutional neural networks as a predictor of realtime ageing biomarkers. Newcastle University.
17. Yaseen, M. (2024). What is YOLOv8: An in-depth exploration of the internal features of the next-generation object detector. arXiv preprint arXiv:2408.15857.
18. Supriya, M., & Prasad, P. K. (2025). Vehicle detection and counting system using OpenCV. International Journal of Progressive Research in Engineering Management and Science, 5(8), 1399–1404.
19. Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. arXiv preprint arXiv:1502.03167.

20. Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. IEEE Transactions on Knowledge and Data Engineering, 22(10), 1345–1359. https://doi.org/10.1109/TKDE.2009.191

AI Use Disclosure

Artificial intelligence tools were used in the preparation of this project report. Specifically, Claude (Anthropic) was used to assist with report structuring, grammar review, and writing feedback during the drafting process. Perplexity was used for literature search and background research.

All technical work, including model implementation, training, evaluation, and analysis, was conducted independently by the author using Google Colab. All results, figures, tables, and code are the original work of the author. AI assistance was limited to the written presentation of ideas, not to the generation of technical content, data, or conclusions.